Date: 1/8/26

Test

Name : P.R.Harshini
Reg No : 192524165
Course Code : CSA1407
Course Name : Compiler Design

1) $E \longrightarrow E+E / id$

ambiguous — more than one production

$E \longrightarrow E+E$
$E \longrightarrow id$

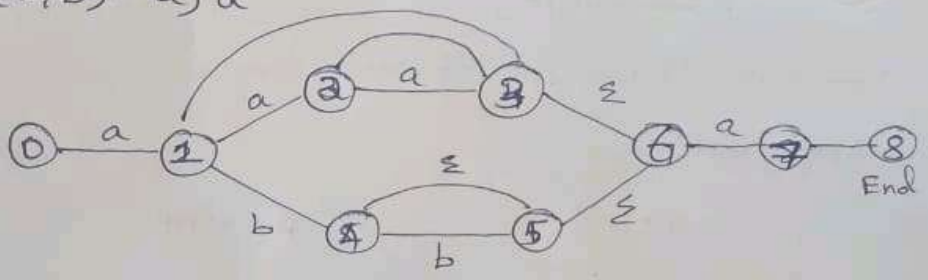
as it contains more than one production it is ambigous grammar.

92/100

V-Good.

2) NFA

$(a\,(a/b)^*\,a)\,a$



Accepting

a a
a a a
a b a
a a a a
a a b a
a a a a a
a b b b a

Rejecting

ε
a
b
b a
a b

3) Parse tree for id + id * id

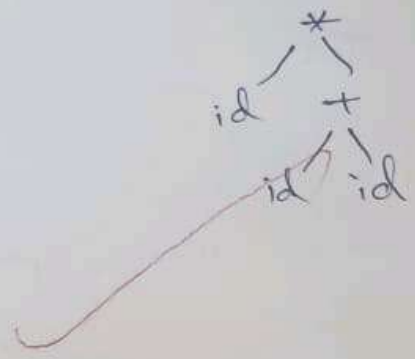
$E \longrightarrow TE'$

$E' \longrightarrow +TE' \mid \varepsilon$

$T \longrightarrow FT'$

$T' \longrightarrow *FT' \mid \varepsilon$

$F \longrightarrow (E) \mid id$



4) Left recursion for $A \longrightarrow A\alpha \mid \beta$

$A \longrightarrow A\alpha$

$A \longrightarrow \beta$

$A \longrightarrow \beta A'$

$A' \longrightarrow \alpha A' \mid \varepsilon$

5) $S \longrightarrow iEts \mid iEtSeS \mid a$ } given

Left factoring:

$S \longrightarrow iEtSS'$

$S' \longrightarrow eS \mid \varepsilon$

$S \longrightarrow a$, $S \longrightarrow iEtSeS$, $S \longrightarrow iEts$ ⇒ three production from given.

$A = \alpha B \beta$

$S = A$, α is considered as 1

6) $S \longrightarrow aBDh$

$B \longrightarrow cC$

$C \longrightarrow bC \mid E$

$C \longrightarrow bC$

$C \longrightarrow \varepsilon$

$C \longrightarrow \varepsilon C'$

$C' \longrightarrow CC' \mid \varepsilon$

$D \longrightarrow EF$

$E \longrightarrow g \mid \varepsilon$

$F \longrightarrow f \mid \varepsilon$

FIRST(S) = {a,

FIRST(B) = {c,

FIRST(C) = {b, ε

FIRST(D) = {g, ε

FIRST(E) = {g, ε}

FIRST(F) = {f, ε}

$A \longrightarrow \alpha B \beta$

FOLLOW(S) = {$, }

FOLLOW(B) = {g, }

FOLLOW(C) = {$, g}

FOLLOW(D) = {$, g}

FOLLOW(E) = {$, g}

FOLLOW(F) = {$, g}

7) $S \longrightarrow Sa \mid Sb \mid cd$

$S \longrightarrow Sa$

$S \longrightarrow Sb$

$S \longrightarrow cd$

$S \longrightarrow cdS'$

$S' \longrightarrow aS' / bS' / \varepsilon$

Yes, it requires a left recursion.

8) $E \longrightarrow TE'$

$E' \longrightarrow +TE' / \varepsilon$

To find FIRST(E')

FIRST(E') $\longrightarrow \{+, \varepsilon\}$

9) $S \longrightarrow (S)S / \varepsilon$

Yes, () is a valid one, because () are considered as Terminals.

They are in the form of $\alpha B \beta \Rightarrow (S)$

10) Actions performed in shift-reduce parsing are:

1) Shift - Current symbol from i/p buffer on to the stack.

2) Reduce - Pops out the RHS and push the LHS of the production on to the stack.

3) Accept - Includes $ symbol, the i/p string is accep

4) Error - if parser cannot perform shift or reduce or state.

11) Find the closure of the following production. (LR) grammar.

LR(0)

$S' \longrightarrow S$

$S \longrightarrow CC$

$C \longrightarrow cC \mid d$

item $(S' \longrightarrow *S) \Rightarrow$ find the closure

$S' \longrightarrow .S$

$S \longrightarrow .CC$

$C \longrightarrow .cC / .d$

The closure of $S' \longrightarrow *S$ is $S' \longrightarrow .*S$

12) i) $S \longrightarrow AB$

$A \longrightarrow a.$

$B \longrightarrow b$

ii) grammar $S \longrightarrow asb \mid ab$

check → aabb is accepted or not

sol: i) FIRST(S) = {a}

FIRST(A) = {a}

FIRST(B) = {b}

sol: ii) $S \longrightarrow asb/ab$

parse cannot be generated hence the string is not accepted.

$S \longrightarrow A$ $\quad A \longrightarrow aB/Ad$

$B \longrightarrow b$ $\quad C \longrightarrow g$

Left Recursion:

$A \longrightarrow aB/Ad$

$A \longrightarrow aB$

$A \longrightarrow Ad$

$A \longrightarrow aBA'$

$A' \longrightarrow dA'/\varepsilon$

FIRST(S) = { a }

FIRST(A) = { a }

FIRST(A') = { d, ε }

FIRST(B) = { b }

FIRST(C) = { g }

$A \longrightarrow \alpha B\beta$

FOLLOW(S) = { $, }

FOLLOW(A) = { $ }

FOLLOW(A') = { $ }

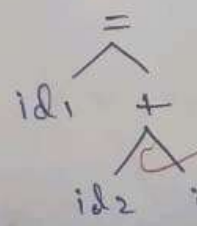
FOLLOW(B) = { d, $ }

FOLLOW(C) = { d, $ }

14) Simple lex code to parse expression with addition.

eg: $a = b + c$

<id1> <=> <id2> <+> <id3> Tokens



Parse tree

$t_1 = id_2 + id_3$

$id_1 = t_1$

intermediate code generation & code optimization.

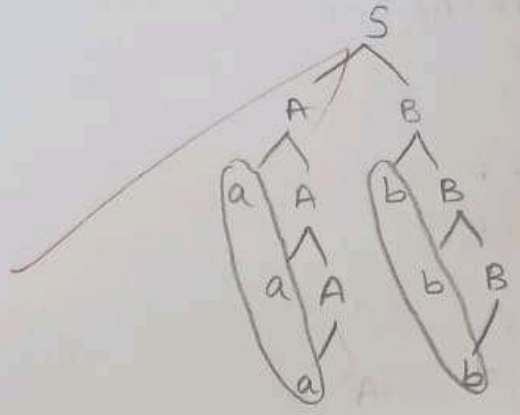
15) Given grammar

$S \longrightarrow AB$

$A \longrightarrow aA|a$

$B \longrightarrow bB|b$

String "aaabbb"



16) $S \longrightarrow AB$

$A \longrightarrow aA|\varepsilon$

$B \longrightarrow bB|c$

find FOLLOW of all Non-terminals.

FIRST(S) — {a, ε}

FIRST(A) — {a, ε}

FIRST(B) — {b, c}

FOLLOW(S) — {b}

FOLLOW(A) — {b}

FOLLOW(B) — {b}

Left recursion:

$A \longrightarrow aA|c$ $\quad B \longrightarrow bB|C$

$A \longrightarrow aA$ $\quad B \longrightarrow bB$

$A \longrightarrow c$ $\quad B \longrightarrow c$

$A \longrightarrow cA'$ $\quad B \longrightarrow cB'$

$A' \longrightarrow AA'|\varepsilon$ $\quad B' \longrightarrow BB'|\varepsilon$

17) Rule to Eliminate Left Recursion and Left factoring:-

Eliminate Left Recursion:

1) Check both LHS & RHS has same Non terminal.

2) Utill it gets to check the Terminal.

3) If same Non-terminal occurs left recursion takes place.

$$A \longrightarrow \alpha B \mid \beta$$

$$A \longrightarrow \alpha B$$
$$A \longrightarrow \beta$$

ii) Follow the same step as left recursion and substitute the value of it.

18) id + id * id

$$F \longrightarrow TE'$$
$$E' \longrightarrow +TE' \mid \varepsilon$$
$$T \longrightarrow FT'$$
$$T' \longrightarrow *FT' \mid \varepsilon$$
$$F \longrightarrow (E) \mid id$$

i)

FIRST(E) $\Longrightarrow$ { (, id }	FOLLOW(E) = { $,) }
FIRST (E') = { +, ε }	FOLLOW(E') = { $,) }
FIRST (T) = { (, id }	FOLLOW(T) = { +, $,) }
FIRST (T') = { *, ε }	FOLLOW(T') = { +, $,) }
FIRST (F) = { (, id }	FOLLOW(F) = { *, +, $,) }

Non-terminal	id	(	)	*	+	$
E	$E \to TE'$	$E \to TE'$				
E'			$E' \to \varepsilon$		$E' \to +TE'$	$E' \to \varepsilon$
T	$T \to FT'$	$T \to FT'$				
T'			$T' \to \varepsilon$	$T' \to *FT'$	$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to id$	$F \to (E)$				

input	string	stack
$	id+ id*id $	—
$E'T	id+id*id $	E → TE'
$E'T'F	id+id*id $	T → FT'
$E'T'id	id+id*id $	F → id
$E'T'	+id*id $	T' → ε
$E'	*id*id $	+TE'
$E'T*	id*id $	E' → ε
$T	id*id $	T → FT'
$FT'F	id*id $	F → id
$T'id	*id $	T' → *FT'
$T'F*	id $	F → id
$T'id	$	T' → ε
$	$	Accept

ii) Shift reduce parser. id > (>) > * > + > E > $

LHS	String	RHS
$	id+id*id $	shift
$id	+id*id $	reduce E → id
$E	+id*id $	shift
$E+	id*id $	shift
$E+id	*id $	reduce E → id
$E+E	*id $	reduce E → E+E
$E	*id $	shift
$E*	id $	shift
$E*id	$	reduce E → id
$E*E	$	reduce E → E*E
$	$	Accept

$E \longrightarrow E+E$

$E \longrightarrow E*E$

$E \longrightarrow (E)$

$E \longrightarrow id$

input string $id_1 + id_2 * id_3$

LHS	string	RHS
$	$id_1 + id_2 * id_3 \$$	Shift
$ id1	$+ id_2 * id_3 \$$	reduce $E \longrightarrow id_1$
$E	$+ id_2 * id_3 \$$	shift
$E+	$id_2 * id_3 \$$	shift
$E+id2	$* id_3 \$$	reduce $E \longrightarrow id_2$
$E+E	$* id_3 \$$	reduce $E \longrightarrow E+E$
$E	$* id_3 \$$	Shift
$E*	$id_3 \$$	Shift
$E*id3	$\$$	reduce $E \longrightarrow id_3$
$E*E	$\$$	reduce $E \longrightarrow E*E$
$	$	Accept

20)
```
#include <stdio.h>
int main() {
    int a=10, b=20;
    int sum = a+b;
    printf("%d", sum);
    return 0;
}
```

a) Lexical Analysis:-

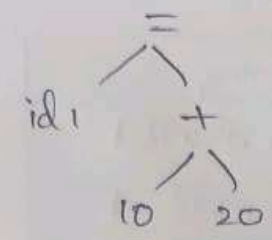
int	b	+
(	=	%
)	10	,
{	;	{
a	20	}
	sum	

5) Lexical Syntax Analyser

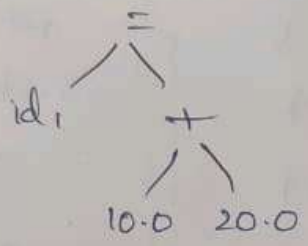
sum = a + b

<id1> <=> <id2> <+> <id3>

<id1> <=> <10> <+> <20>



Semantic Analyser



Intermediate code generator

$t_1 = id_2 + id_3$

$id_1 = t$

code optimization

$t_1 = id_2 + id_3$

$id_1 = t$

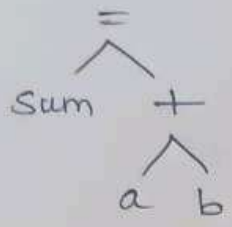
code generation :

LDF R_1, id_3

ADDF R_1, R_1, id_2

STF R_1.

Sum = a+b



c) Sum = a+b